Programação Orientada a Objetos

Encapsulamento

Professor: Pedro Montoaneli
pedromontoaneli@teiacoltec.org

Sumário

- Encapsulamento
  - Membro de Dados
  - Especificadores de Acesso
  - Funções get() e set()
  - Exemplo 1: Classe “Disciplina” com funções get() e set()
  - Exemplo 2: Classe “Disciplina” com matricula
  - Exemplo 3: Acrescentando carga horária
  - Exercícios

Membro de Dados

- Um membro de dados é uma variável declarada **dentro** da definição da classe.
- Os membros de dados são declarados fora da definição dos métodos.
- Um membro de dados possui validade ao longo de todo o escopo da classe, então pode ser declarado tanto no início quanto no fim.

Especificadores de Acesso

- Toda variável ou função declarada dentro de uma classe deve possuir um **especificador de acesso**.
- O especificador de acesso define de a grandeza pode ou não ser acessada diretamente pelo usuário.
- Se a grandeza pode ser acessada, ela é declarada após o rótulo `public:`, caso o contrário, deve-se usar o rótulo `private:`
- Em geral, os membros de dados são declarados como `private:`

Funções `get()` e `set()`

- O acesso/modificação de atributos privados é feito usando funções `get()` e `set()`.

- A função `get()` deve ser do mesmo tipo que o membro de dados e retorna o seu valor.

```
tipoMembro getNomeMembro(){
        return NomeMembro;
}
```

- A função `set()` deve ser do tipo void e modifica/valida o valor do membro.

```
void setNomeMembro(tipoMembro NovoNome){
        NomeMembro = NovoNome;
}
```

Exemplo 1: Classe “Disciplina” com membros e funções `get()` e `set()`

```
1.#include <iostream>
2.#include <string>
3.using namespace std;
4.
5.// Definição da Classe
6.class Disciplina {
7.public:
8.
9.    string getNomeDisciplina(){
10.        return nomeDisciplina;
11.    } // Função get
12.
13.    void setNomeDisciplina(string nome){
14.        nomeDisciplina = nome;
15.    } // Função set
16.
17.    void mensagem(){
18.        cout << "Bem vindos à disciplina de " << getNomeDisciplina() << "!" << endl;
19.    }
20.
21.private:
22.    string nomeDisciplina; // Membro de Dados privado
23.}; // Fim da Classe
```

Exemplo 1: Classe “Disciplina” com membros e funções get() e set()

```
24.int main() {
25.    string nomeDisciplina;
26.    Disciplina poo;
27.
28.    cout << "Digite o nome da disciplina:" << endl;
29.    getline(cin,nomeDisciplina); // Lê a string com espaços em branco
30.    poo.setNomeDisciplina(nomeDisciplina);
31.    poo.mensagem();
32.
33.    return 0;
34.}
```

Exemplo 2: Classe “Disciplina” com matrícula

```
1.#include <iostream>
2.#include <string>
3.using namespace std;
4.
5.class Disciplina {
6.private:
7.    string nomeDisciplina;
8.    int vagas;
9.    int matriculados;
10.public:
11.    void inicializar(string nome, int totalVagas) {
12.        nomeDisciplina = nome;
13.        if (totalVagas > 0) {
14.            vagas = totalVagas;
15.        } else {
16.            vagas = 0;
17.        }
18.        matriculados = 0;
19.    }
```

```
20.    void matricular() {
21.        if (matriculados < vagas) {
22.            matriculados++;
23.        } else {
24.            cout << "Turma lotada." << endl;
25.        }
26.    }
27.    void cancelarMatricula() {
28.        if (matriculados > 0) {
29.            matriculados--;
30.        }
31.    }
32.    int getMatriculados() {
33.        return matriculados;
34.    }
35.    // Resto igual
36.};
```

Exemplo 3: Acrescentando carga horária

```
// novo atributo na parte private
    int cargaHoraria;

// novos metodos na parte public
    void setCargaHoraria(int horas) {
        if (horas > 0) {
            cargaHoraria = horas;
        } else {
            cout << "Carga horaria invalida, valor ignorado." << endl;
        }
    }

    int getCargaHoraria() {
        return cargaHoraria;
    }
```

```
void inicializar(string nome, int totalVagas) {
        nomeDisciplina = nome;
        setCargaHoraria(60);   // valor inicial passando pela validacao
        // Resto igual
    }
```

Exercícios

Exercício 1:

Crie uma classe `Produto` com dois atributos privados, nome, do tipo `string`, e `preco`, do tipo `float`. Forneça `getNome, setNome, getPreco` e `setPreco`, além de um inicializar que receba nome e preço. O `setPreco` deve recusar valores negativos, mantendo o preço anterior e avisando o usuário. No `main`, crie dois produtos, tente definir um preço negativo em um deles, e imprima o nome e o preço dos dois no fim.

Exercício 2:

Crie uma classe `Cofre` com um atributo privado total, do tipo `float`, que representa o dinheiro guardado. A classe deve ter inicializar, que começa o total em zero, um método `guardar(float valor)` que só aceita valores positivos, um método `retirar(float valor)` que só deixa retirar se houver saldo suficiente, avisando quando não houver, e um `getTotal`. No `main`, faça algumas operações, incluindo uma retirada maior que o total, e imprima o valor final.

Exercício 2:

Crie uma classe `Cofre` com um atributo privado total, do tipo `float`, que representa o dinheiro guardado. A classe deve ter inicializar, que começa o total em zero, um método `guardar(float valor)` que só aceita valores positivos, um método `retirar(float valor)` que só deixa retirar se houver saldo suficiente, avisando quando não houver, e um `getTotal`. No `main`, faça algumas operações, incluindo uma retirada maior que o total, e imprima o valor final.